#Unification in first order logic

```
# Week 7: First Order Logic
# Implementation of Unification Algorithm

# Check whether a term is a variable
def is_variable(x):
    return isinstance(x, str) and x.islower()


# Apply substitutions to a term
def apply_substitution(term, subst):

    if isinstance(term, str):
        while term in subst:
            term = subst[term]
        return term

    if isinstance(term, tuple):
        return tuple(apply_substitution(x, subst) for x in term)

    return term


# Check whether variable occurs in a term
def occurs(var, term, subst):
```

```
    term = apply_substitution(term, subst)

    if var == term:
        return True

    if isinstance(term, tuple):
        return any(occurs(var, x, subst) for x in term)

    return False


# Unification function
def unify(x, y, subst=None):

    if subst is None:
        subst = {}

    x = apply_substitution(x, subst)
    y = apply_substitution(y, subst)

    # If both are identical
    if x == y:
        return subst

    # If x is a variable
    if is_variable(x):

        if occurs(x, y, subst):
```

```
        return None

    subst[x] = y
    return subst

# If y is a variable
if is_variable(y):

    if occurs(y, x, subst):
        return None

    subst[y] = x
    return subst

# Check whether both are compound expressions
if isinstance(x, tuple) and isinstance(y, tuple):

    # Check predicate symbols
    if x[0] != y[0]:
        return None

    # Check number of arguments
    if len(x) != len(y):
        return None

    # Unify arguments one by one
    for i in range(1, len(x)):
```

```
            subst = unify(x[i], y[i], subst)

            if subst is None:
                return None

        return subst

    # Otherwise unification fails
    return None



# Main program
expr1 = ("Eats", "x", "Apple")
expr2 = ("Eats", "Riya", "y")

print("Expression 1:", expr1)
print("Expression 2:", expr2)

result = unify(expr1, expr2)

if result is not None:
    print("\nUnification Successful")
    print("Substitution:", result)
    print("Unified Expression:",
          apply_substitution(expr1, result))
else:
    print("\nUnification Failed")
```

```
Expression 1: ('Eats', 'x', 'Apple')
Expression 2: ('Eats', 'Riya', 'y')

Unification Successful
Substitution: {'x': 'Riya', 'y': 'Apple'}
Unified Expression: ('Eats', 'Riya', 'Apple')
```

: